Data Science and AI Diploma

Introduction to Data Structures in Python

Session 3

AMIT

Introduction to Data Structures

Data structures help organize and store data efficiently.

Python provides several built-in data structures:

Lists
Tuples
Sets
Dictionaries (covered separately)
Each has unique properties and use cases.

Lists in Python

Definition: A list is an ordered, mutable collection of items.

Characteristics:

- Can contain different data types.
- Supports indexing and slicing.
- Allows modifications (adding/removing elements).

Example:

```
my_list = [1, "apple", 3.14, True]
print(my_list[1])  # Output: apple
```

List Indexing & Slicing

Accessing elements using indexing:

```
my_list = [10, 20, 30, 40]
print(my_list[0])  # Output: 10
print(my_list[-1]) # Output: 40
```

Slicing lists:

```
print(my_list[1:3])  # Output: [20, 30]
print(my_list[:2])   # Output: [10, 20]
```

List Operations

Adding elements: append(), insert(), extend()

```
my_list.append(50)  # Adds to the end
my_list.insert(1, 15)  # Inserts at index 1
```

Removing elements: remove(), pop()

```
my_list.remove(30)  # Removes first occurrence
my_list.pop(2)  # Removes item at index 2
```

Sorting and reversing: sort(), reverse()

```
my_list.sort()  # Sorts in ascending order
my_list.reverse()  # Reverses the list
```

Tuples in Python

- Definition: A tuple is an ordered, immutable collection of items.
- Characteristics:
  - Cannot be modified after creation.
  - Faster than lists.
  - Can be used as dictionary keys.

```
my_tuple = (1, "banana", 3.14)
print(my_tuple[1])  # Output: banana
```

Tuple Operations

Accessing elements: Similar to lists.

```
print(my_tuple[0])  # Output: 1
```

Concatenation & repetition:

```
new_tuple = my_tuple + (10, 20)
print(new_tuple)  # (1, "banana", 3.14, 10, 20)
```

Converting tuple to list:

```
my_list = list(my_tuple)
```

Sets in Python

Definition: A set is an unordered collection of unique elements.

Characteristics:

- No duplicate values.
- Supports mathematical operations.
- Unordered and unindexed.

Example:

```
my_set = {1, 2, 3, 4, 4, 5}
print(my_set)  # Output: {1, 2, 3, 4, 5}
```

Set Operations

Adding and removing elements:

```
my_set.add(6)  # Adds an element
my_set.remove(3)  # Removes an element
```

Mathematical set operations:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 | set2)  # Union: {1, 2, 3, 4, 5}
print(set1 & set2)  # Intersection: {3}
print(set1 - set2)  # Difference: {1, 2}
```

When to Use Lists, Tuples, or Sets?:

Data Structure	Ordered?	Mutable?	Duplicates Allowed?	Use Case
List	✅ Yes	✅ Yes	✅ Yes	General-purpose storage
Tuple	✅ Yes	❌ No	✅ Yes	Fixed data, faster lookup
Set	❌ No	✅ Yes	❌ No	Unique values, mathematical ops

Summary

- Lists: Ordered, mutable, supports duplicates.
- Tuples: Ordered, immutable, supports duplicates.
- Sets: Unordered, mutable, unique elements.
- Choose the right data structure based on the use case!